Secure Software Engineering

What is Software Engineering?

Misconceptions: 70% of projects fail, Most games = 6M lines of code, large and complex

Fault: occurs when a human makes a mistake in performing some software activities

Failure: is a departure from the system's required behavior

Examples:

- IRS hired Sperry Corporation to build an automated federal income tax form processing process, extra 90 mill overbudget, lost 62.5 mill over refunds
- Malfunctioning code in Therac-25 killed several people

Software engineering is the construction and management of large software projects

Consists of methods, tools, procedures, philosophy/approach

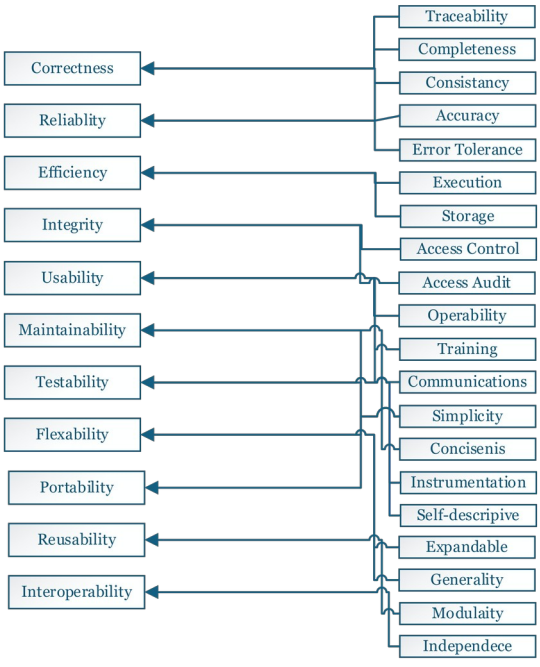
Analysis: decompose a large problem into smaller, understandable pieces, abstraction

Synthesis: build (compose) software from smaller building blocks

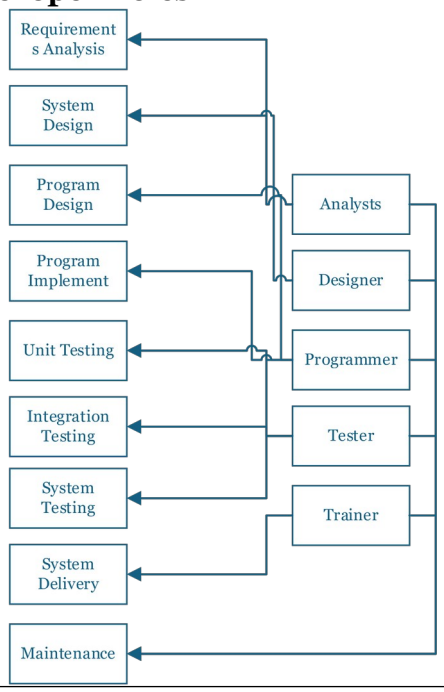
Good software: quality product, quality process, business environment

Building a System: Requirement analysis and definition • System design • Program design • Writing the programs • Unit testing • Integration testing • System testing • System delivery • Maintenance

McCall's quality model:



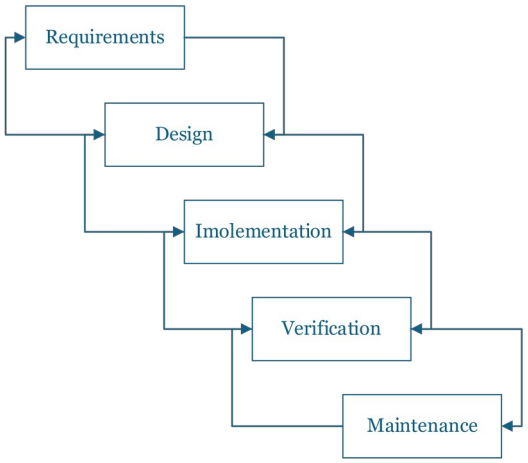
Delevoper Roles



Software Models

Software process model: is an abstract representation of a process. It presents a description of a process from some perspective.

Waterfall Model



traditional, linear, and sequential software development methodology non-iterative approach, where changes are difficult to accommodate once a phase is completed

Characteristics

- Sequential Flow
- Phase Dependency
- Documentation-Driven
- Minimal Customer Involvement

Advantages

- Simple and easy to understand.
- Clearly defined stages and milestones.
- A well-documented process, useful for teams with less experience.
- Suitable for projects with well-defined, stable requirements

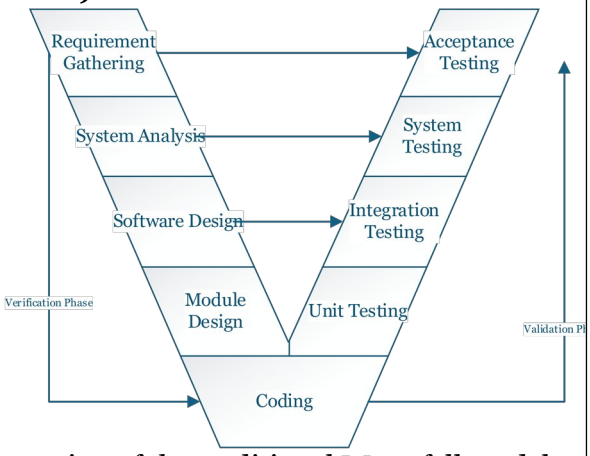
Disadvantages

- Inflexibility
- Late Testing
- Minimal Customer Feedback
- Unsuitable for Complex or Long-Term Projects

Best For

- clear and unchanging requirements
- Short-duration projects
- Systems where extensive documentation is required

V-Model(Verification and Validation Model)



extension of the traditional Waterfall model and is widely used in industries where high-quality and reliable software is crucial.

Characteristics

- Sequential Process
- Verification and Validation
- Early Testing

Advantages

• Clear and structured process
• Early detection of defects through parallel development and testing
• Well-suited for projects with clearly defined requirements

Disadvantages

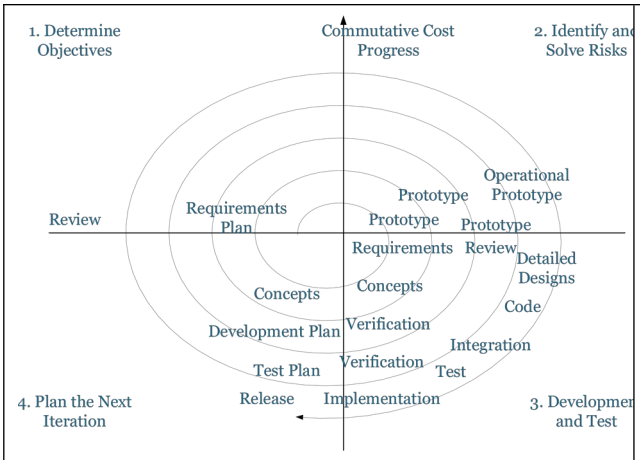
• Not flexible for changing requirements
• Costly and time-consuming for iterative or agile projects

Best For

- requirements are well understood
- strict quality assurance is required

Spiral model

Agile model



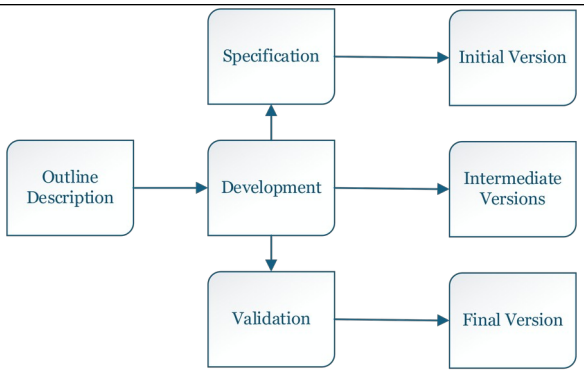
Created by Boehm in 1988, combines development activities with risk management to minimize and control risks

Characteristics - four major activities

- Plan
- Determine goals, alternatives and constraints
- Evaluate alternatives and risks
- Development and test

Best For

• For large, high-risk projects.
• When requirements are unclear or expected to evolve.
• When customer feedback is critical to project success.



An iterative and incremental approach for S/W development

Two versions

- **Scrum**

Works in a series of sprints 1–4-week duration
Scrum Master helps maintain team habits
Starts with a sprint planning meeting to select high priority items from the product backlog.
Called sprint backlog
Dev team works on items in the spring backlog
Daily Scrum short standup meeting
Regular review of the sprints having demos
Examination of what went well/improved
End of sprint items are packed for realize
Incomplete items are returned to the backlog

- **Kanban**

Continuous process
Agile coach maintains team habits
Items are pulled from the product backlog
Each Item has a work in progress limit (WIP)
Empty column indicates sending a new item
Daily standup meeting
Each item is packaged for release
Demonstrations for stakeholders
A look at what went well to aim to improve

Best For

Evolving requirements
Frequent feedback
Quick and incremental delivery
Collaborative teams

Software Requirements

Requirement: objects or entities • the state they can be in • change states or characteristics

Requirements Process: ask questions, capture in a model, specification/design, validation

Types of requirements

- **Functional requirement:** describes required behavior in terms of required activities
- **Quality requirement:** describes some quality characteristic that the software posses
- **Design constraint:** a design decision such as choice of platform/components
- **Process constraint:** a restriction on the techniques or resources to build

Metrics for nonfunctional requirements

- **Speed:** transactions per second, response time, screen refresh time
- **Size:** MB, number of ROM chips
- **Ease of use:** time to failure, probably of unavailability, rate of occurrence
- **Robustness:** time to restart after failure, possibility of corruption
- **Portability:** number of target systems, percentage of target dependent statements

Characteristics of requirements

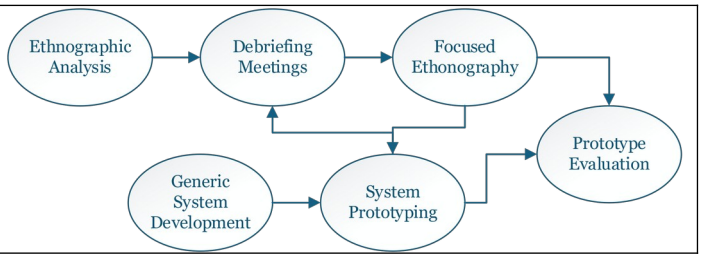
- Correct
- Consistent: no conflicts
- Unambiguous: fit criteria
- Complete: all possible states
- Feasible: not impossible to implement
- Testable: fit criteria
- Traceable: organized, links between definition and specification

UML (Unified Modeling Language): used to document software specifications and designs

Prototyping: Visualize a proposed system, Throwaway approach/Evolutionary approach

Analysis process: Discovery→classification/organization→priorities/negation→specification

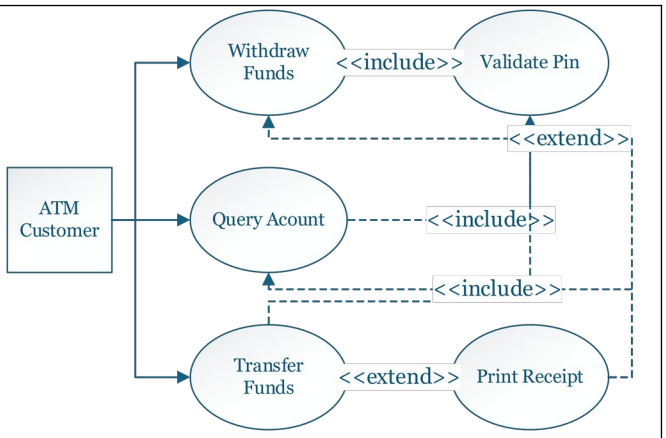
Ethnography: A social scientist spends time observing/analyzing how people work

Analysis process: Discovery→classification/organization →priorities/negation→specification **Ethnography:** A social scientist spends time observing/analyzing how people work	

Ways of writing a system requirements: natural language, Structured natural language, design description languages, graphical notations, mathematical specifications

Requirement Document: Preface, Introduction, glossary, user requirements definition, system architecture, system specifications, system models, system evolution, appendices, index

Use Case Modeling

Dependency relationships: **<<Include>> relationship**: use case depends on another use case to for its functionality a behavior that is shared among multiple use cases, indicated by a dashed arrow **<<extend>> relationship**: use case adds new behavior to another use case optional and can be included or not depending on conditions, indicated by a solid arrow	

ACTORS: An actor is anyone or anything that must interact with the system. • Users • External systems • Sensors • Timer • APIs

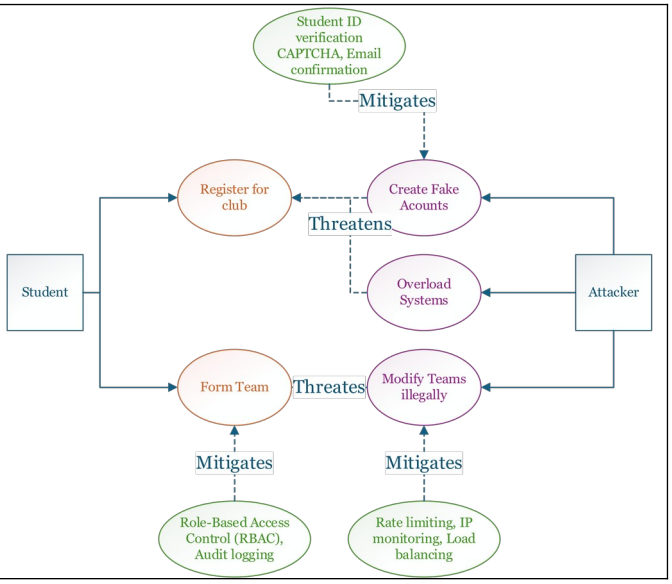
USE CASES: description of how users interact with a system to achieve specific goal/task

Primary actors: initiate a use case and hence are somewhat independent

Secondary actors, used by the system but do not interact with the system on their own

CONTENTS OF A USE CASE NARRATIVE: Title, description, dependency, preconditions, basic flow, alternate flows, post-conditions

Misuse Cases

Misuse case: type of analysis on how a system can be exploited, misused, attacked **Steps:** Know your enemy, Carry out relevant attacks, Design countermeasures, Carry Out Relevant Attacks, Enhance Security, Improve System Resilience, Compliance/Regulations	

Security Requirements

1. Requirements Analysis – Security Requirements, Misuse Cases

2. Design – Risk Analysis, Vulnerability Mapping

3. Construction – Secure Coding Practices, Code Review

4. Testing – Penetration Testing, Risk-based Security Test

5. Installation – Final Security Review

6. Maintenance – Periodic Security Review and Updates

Gathering Requirements: functional requirement, nonfunctional requirement

Fail case: This is what will happen if the requirement is not fulfilled during operation

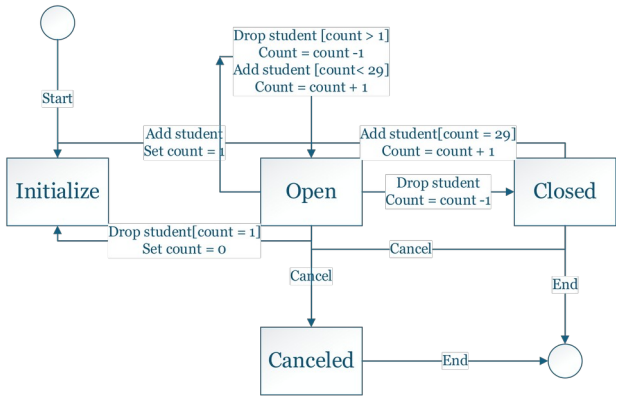
Consequence of failure: This is the result of the fail case

Associated risks: The associated risks include sensitive information that is revealed

State Transition Diagram

models the system behavior in response to events that cause a transition from one state

State: recognizable situation, over an interval of time, interval between successive events
Event: causes change of state, at a point on time, no duration
Condition: Boolean expression, when event occurs, condition must be true to occur
Action: executed because of state transition, entry action, exit action, optional
Transition: Event [condition] / action(s)

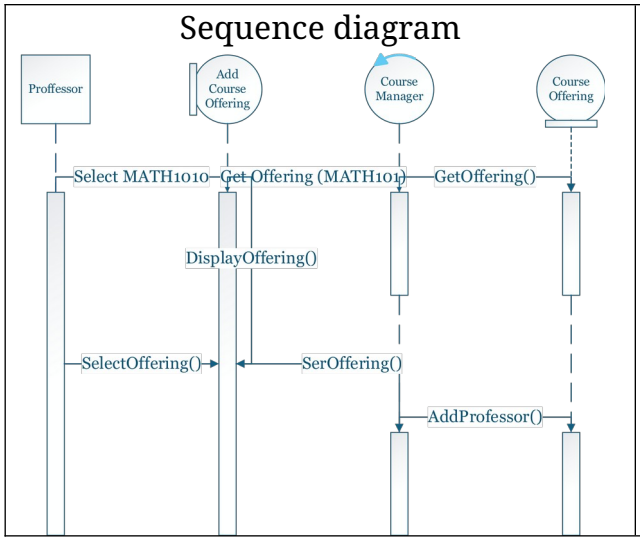


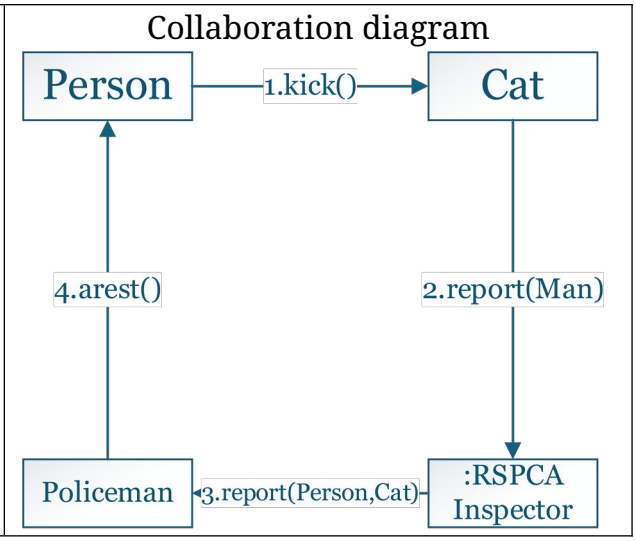
Interaction Diagram

Scenario: is an instance of a use case, describe how use cases are realized

• What objects are needed for the flow, What messages the object need to send

• What actors initiate flow, The order of messages





Software Architecture

Software architecture high-level structure of a software system, defining components interact/function together, blueprint for both the system and the project

Aspect	Software Architecture	Software Design
Scope	High-Level system	Low-level design
Components	Services, layer, databases	Functions/classes/ modules
Decision Making	System-wide decisions	Implementation choices
Output	Architectural diagrams	Class diagrams, algorithms

Explicit architecture: Stakeholder communication, System analysis, Large-scale reuse

Characteristics: Performance security, safety, availability, maintainability

Each architectural model only shows one view or perspective of the system

---

4 + 1 view model: four concurrent views, plus an additional "plus one" view for use cases

A logical view, which shows the key abstractions in the system as objects or object classes.

A process view, which shows how, at run-time, the system is composed of processes.

A development view, which shows how the software is decomposed for development.

A physical view shows system hardware and software components distributed processors

Related use cases or scenarios (+1)

Layered Architecture: Organizes system into hierarchical layers where each layer serves a

distinct purpose. Includes Presentation, Business Logic, Data Access, and Database layers

Pros: maintainability, scalability, development layers, reusable components for efficiency

Cons: Performance overhead multiple layers of processing, limited flexibility components

---

Client-Server Architecture: Divides system into client and server components, client requests services, and the server processes and responds, web applications/distributed

Pros: centralized management improved security/control, scalability by server instances

Cons: server dependency cause bottlenecks, requires network connectivity

---

Two-tier: layers are mapped onto two computer systems

three-tier: layers are mapped onto three computer systems

	2-tier	3-tier
Scalability	Less scalable	More scalable with requests
Security	Less secure	Highly secured
Flexibility	Less flexible	More flexible, less coupled
Maintainability	Easy	More difficult

Thin-client model: processing/data management is carried out on the server

Fat-client model: server is only responsible for data management

	Thin client	Thick client
Local resources	Consume few resources	Consume more resources
Security	Less security threats	More security threats
Network latency	fast internet need	slow internet need

Deployment	Easy to deploy	Expensive to deploy
Data validation	At the server side	At the client side

MVC (Model-View-Controller): Separates into three interconnected components

- Model: Manages application data and business rules.
- View: Presents data to the user and handles UI rendering.
- Controller: Handles user input and updates Model and View accordingly

Pros: Allow parallel development, improves maintainability by separating concerns

Cons: introduce complexity, tight coupling between components make change difficult

---

Pipe-and-Filter Architecture: Data flows through a series of independent components

- Each filter performs a specific transformation or computation on the data
- Commonly used in data transformation, compiler design, and multimedia processing

Pros: encourages modularity/reusability of components, parallel/distributed processing

Cons: high overhead due to intermediate data transfer, complex debugging/error handling

---

Peer-to-Peer (P2P) Architecture: no central server, nodes interact each other for resources

- Peers act both as clients and servers, distributing workloads dynamically
- Used in file-sharing, blockchain, and decentralized computing applications

Pros: highly scalable/fault-tolerant, no central point of failure

Cons: difficult to manage and secure, result in inconsistent data availability

---

Choosing the Right Architecture

- Layered: Best for enterprise applications requiring separation of concerns.
- Client-Server: Ideal for web-based systems needing centralized control.
- MVC: Suitable for interactive applications with user interfaces.
- Pipe-and-Filter: Best for data transformation and parallel processing.
- P2P: Works well for decentralized and distributed applications.

Class Diagram

Class Diagram defines: the classes of objects in the system, the attributes and operations of the classes, and the relationships between classes.

Classes: group of objects with common properties (attributes), and common behavior (operations), in the UML it is represented as a compartmentalized rectangle

Association: defines a relationship between two or more classes.

Multiplicity of an Association: how many instances of one class relates to other classes

Association Class: is a class that models an association between two or more classes

Association: general, undirected relationship between two classes, indicating that they are related in some way

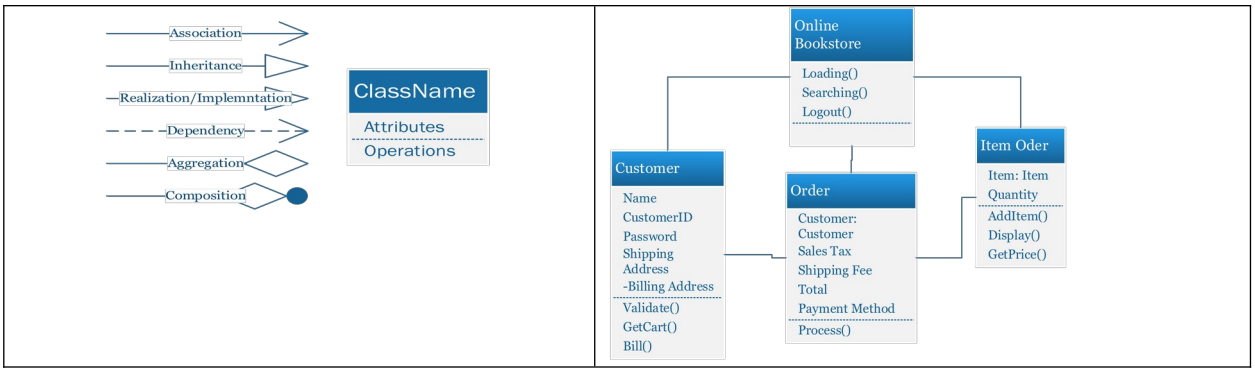
Inheritance: "is-a" relationship where one class (subclass or derived class) inherits properties and behaviors from another class

Implementation: one class implements an interface or abstract class. It indicates that the class provides the concrete implementation of the interface's methods.

Dependency: temporary or weak relationship where one class relies on another for a specific task or functionality

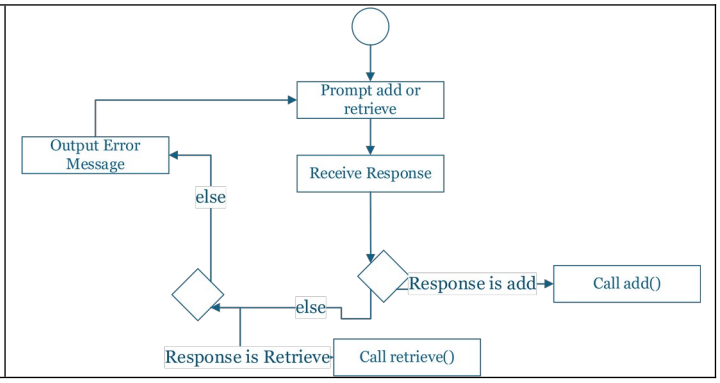
Aggregation: "has-a" relationship where one object contains another object, but the contained object can exist independently of the container.

Composition: stronger "has-a" relationship than aggregation, where one object contains another, and the contained object cannot exist without the container.



Activity Diagram

Activity Diagram: is another way to model the flow of events.
Parts of Activity Diagram: Activity, Start and End States, Transitions, Decision and guard conditions, Synchronization



Software Design

Design is to produce a solution to a problem:

- Set of documents on whose basis software can be fully programmed
- The problem: requirements specification
- The solution: your description of how the requirements is to be met

Goals of software design

Correctness: functions precisely as intended, without errors/deviations from specifications

Modularity: separate the various unrelated aspects of a system, each aspect is in isolation

Coupling: How tightly coupled modulars are with each other

Cohesion: degree to which the elements inside a modular belong together

Robustness: ability to handle anomalous situations even in the presence of errors

Flexibility: implies a design that can easily accommodate the changes
Reusability: components of a developed system can be used again in new applications
Efficiency: execute required functionality within required time constraints, time/space
Reliability: application is reliable if it is relatively defect free
Usability: has high usability if users find it easy to use, attained through design
Principles of class design
Open Closed Principle: module should be open for extension but closed for modification,
change what the modules do, without changing the code of the modules, abstraction
Liskov Substitution Principle: Subclasses should be substitutable for their base classes
If its preconditions are no stronger than the base class method.
If its postconditions are no weaker than the base class method.
Derived methods should expect no more and provide no less
Dependency Inversion Principle: Depend upon abstractions. Do not depend upon concretions. Concrete things change a lot, abstract things change much less frequently
Interface Segregation Principle: Many clients specific interfaces are better than one general
purpose interface. Create specific interfaces for each client and multiply inherit to class

Fat Service with Integrated Interfaces: Whenever a change is made to one of the methods
that ClientA calls, ClientB and ClientC may be affected. Recompile might be necessary.
Segregated Interfaces: If the interface for ClientA needs to change, ClientB and ClientC will
remain unaffected. They will not have to be recompiled or redeployed.

Package cohesion and coupling:

Common Closure Principle: Classes that change together, belong together
Common Reuse Principle: Classes that aren't reused together not be grouped together
CCP large packages easy for reusers, CRP small packages easy for maintainers

Acyclic Dependencies Principle: dependencies between packages must not form cycles
Stable Dependencies Principle: dependencies between packages should be stable
Stable Abstractions Principle: Stable packages should be abstract packages

Design patterns are class combinations and accompanying algorithms that fulfill purposes
Viewpoints – ways to describe patterns
Levels – decomposition of patterns
Roles – the "players" in pattern usage

Facade Design Pattern: simplifies complex systems by providing a single interface
Adapter Design Pattern: usage of an existing class's interface as an additional interface

Principles of Security and Quality

keeping C.I.A: Confidentiality, Integrity and availability

Secure Code: Authorization, Authentication, Secure the Weakest Link, Defense in Depth, Fail Securely, Least Privilege, Keep It Simple, Secrets Are Not Kept, Complete mediation, Separation of privilege, Server-Side Validation

Types of Attacks

INSERT into employee (Name, EID, Password) Values (Ryan', 'EID5', 'pasd12');

SELECT * FROM employee WHERE EID='EID5';

UPDATE employee SET Password='admin' where Name=''Ryan'

Sql injection example:

Select name, Salary From employee WHERE eid =’ USER INPUT’ and Pass = ‘INPUT’;

Malicious code eid= EID5 # which ignores the password requirement through comment

Buffer Overflow occurs when a program tries to write too much data in a fixed length block of memory (a buffer)

What happens if you enter a username that is longer than 8 characters? What will happen if we run the program and enter a very long string, e.g. 5 million characters? Data is put on the stack anyhow, but it overwrites current data on said stack

One common approach is to fill as much of the program's memory space as possible with no-op ("no operation") instructions then place the injected code at the end. If the program's execution context lands on any of the no-op codes, it will skip to the next no-op, then the next, and so on, until it finally executes the injected code

Most languages used to write web-code - like Python, Ruby, Node, Java and .NET - used "managed memory", and are immune to buffer overflow attacks.

Cross Site Request Forgery (CSRF)

When a request is sent to example.com from a page coming from example.com, the browser attaches all the cookies belonging to example.com. Now, when a request is sent to example.com from another site (different from example.com), the browser will attach the cookies too. Because of above behavior of the browsers, the server cannot distinguish between the same-site and cross-site requests It is possible for third-party websites to forge requests that are the same as the same-site requests.

Cross-site scripting (XSS) XSS vulnerabilities permit a malicious user to execute arbitrary chunks of JavaScript when other users visit your site. In XSS, an attacker injects his/her malicious code into the victim’s browser via the target website. When code comes from a website, it is considered as trusted with respect

to the website, so it can access and change the content on the pages, read cookies belonging to the website and send out requests on behalf of the user.

Software Testing

Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.

The first goal leads to validation testing. You expect the system to perform correctly using a given set of test cases that reflect the system's expected use.

The second goal leads to defect testing. The test cases are designed to expose defects. The test cases in defect testing can be deliberately obscure and need not reflect how the system is normally used.

V & V: confidence that the system is 'fit for purpose means verification and validation

Meets software purpose, user expectations and marketing environment.

Unit testing is the process of testing individual components in isolation

Test case template

Test Case ID	[Unique Identifier for the Test Case]
Test Case Title	[Short and descriptive title of the test case]
Test Case Description	[A brief description of what the test case will validate]
Test Case Priority	[High/Medium/Low]
Pre-Conditions	[conditions that must be met before the test can be executed]
Test Data	[Specific data to be used in the test]
Test Steps	[Step-by-step instructions for executing the test]
Expected Results	[What the system should do or show after executing the test]
Actual Results	[What happened when the test was executed]
Pass/Fail	[Indicate whether the test passed or failed]
Post-Conditions	[Describe the state of the system after the test]
Tested By	[Name of the tester executing the test]
Test Data	[Date when the test was executed]
Remarks	[Additional notes, if any, e.g., environment, issues

	encountered]

Partition testing: A testing technique that divides input data into partitions to reduce the number of test cases.

Equivalence partitions: Groups of input data that are treated the same by the system, used to minimize test cases.

Use-case testing: A method of testing based on the use cases of the system, focusing on user interactions.

Component testing: Testing individual components of a system in isolation to ensure they function correctly.

System testing: Testing the complete and integrated software to verify that it meets specified requirements.

System and component testing: A combined approach to testing both individual components and the entire system.

Test-driven development: A software development process where tests are written before the code itself.

Regression testing: Testing existing software to ensure that new changes have not adversely affected existing functionality.

Release testing: Final testing conducted before the software is released to ensure it is ready for deployment.

Release testing and system testing: Testing encompasses both the final release checks and the overall system functionality.

Performance testing: Testing to determine how a system performs in terms of responsiveness and stability under a particular workload.

User testing: Testing conducted with actual users to validate the usability and functionality of the software.

Alpha testing: Internal testing conducted by developers to identify bugs before releasing the software to external testers.

Beta testing: Testing conducted by a limited number of external users to gather feedback before the final release.

Acceptance testing: Testing to determine if the software meets the acceptance criteria and is ready for delivery.

Code Review: A systematic examination of computer source code to identify bugs and improve code quality.

Integration Testing: Testing the interaction between integrated components to ensure they work together as expected.

Regression Testing: Re-testing after changes to ensure that existing functionality remains unaffected.

Function Testing: Testing to verify that each function of the software operates in conformance with the requirement specification.

Performance Tests: Tests designed to evaluate the performance characteristics of the software under various conditions.

Acceptance Tests: Tests conducted to determine if the software meets the business requirements and is acceptable for delivery.

Pilot test: installation on an experimental basis: A trial run of the software in a real-world environment to evaluate its performance before full deployment.

Alpha test: in-house test: Testing performed by internal staff to identify issues before external testing.

Beta test: customer pilot: A phase where the software is released to a limited audience outside the company for feedback.

Parallel testing: the new system operates parallel with the old system: Running both the new and old systems simultaneously to compare results and ensure accuracy.

Test Documentation: Documentation that outlines the testing process, including test plans, cases, and results.

Software Change and Evolution

Change is inevitable, Systems MUST be changed if they are to remain useful

Majority of the software budget in large companies is devoted to changing and evolving existing software rather than developing new software
Evolution: stage in a software system's life cycle where it is in operational use and is evolving as new requirements are proposed and implemented in the system

Service: the software remains useful, but the only changes made are those required to keep it operational, i.e. bug fixes and changes to reflect changes in the software's environment. No new functionality is added.

Phase-out: The software may still be used but no further changes are made to it.

Urgent changes may have to be implemented without going through all stages of the software engineering process

Software maintenance: Modifying a program after it has been put into use.

Objective of maintenance: Modify existing software while preserving its integrity
Modification requests workflow: Request is logged and tracked The impact of proposed changes is determined Code and other software artifacts are modified Testing is conducted New version of the software product is released

Need for Maintenance Correct faults Improve the design Implement enhancements (perfecting existing functions) Interface with other software Identifying security threats and fixing security vulnerabilities Adapt programs so that different hardware, software, system features, and telecommunications facilities can be used Migrate legacy software

Types of maintenance: Maintenance to repair software faults, Maintenance to adapt software to a different operating environment, Maintenance to add to or modify the system's functionality